Telecom Fraud Radar : Full Step Tracker

Day 1 : Environment Setup & Foundations

Toolchain

1. **Install Docker Engine + Compose**
2. **Install Python 3.13 (venv)**
3. **Install AWS CLI v2**
4. **Install** `awslocal` **in venv**
5. **Remove broken Snap AWS stub**

LocalStack

6. **Write** `docker-compose.yml`
7. **Create** `.env` **with auth token**
8. **Create** `.gitignore` **(protect** `.env`**)**
9. **Launch LocalStack container**
10. **Verify health endpoint (edition: pro)**
11. **Verify** `awslocal` **LocalStack**

Infrastructure Provisioning

12. **Create S3 bronze bucket** (`telco-fraud-bronze`) **YOU ARE HERE**
13. **Create 7 raw prefixes in bronze bucket**
14. **Provision DynamoDB tables** (`subscriber_profile`, `bundle_catalog`)
15. **Provision Kinesis streams** (`sim-activation-stream`, `bundle-purchases-stream`)
16. **Provision SNS topic** (`fraud-alerts-topic`)

Contracts

17. **Write Pydantic schemas** for sources 1, 2, 3, 5, 8
18. **Write JSON schema files** in contracts/

Project Files

19. **Write** README.md (honest LocalStack + simulated data notice)
20. **Write** requirements.txt
21. **Write** Makefile with targets: up, infra, simulate, test, destroy

→ **Day 1 complete.**

Week 2 : Simulator & Bronze Ingestion

22. Configuration & Constants

1. simulator/config.yaml : seed, fraud rate, population size, overlap ratio, output paths
2. simulator/constants.py : countries, bundle catalog seeds, segments, channel weights

Data Generation (the core)

3. simulator/baseline.py : realistic legit behavior (log-normal amounts, Poisson rates, weekday/hour seasonality, channel weights)
4. simulator/fraud_injector.py : 4 fraud archetypes (SIM swap takeover, IRSF, subscription fraud, bundle abuse) with probabilistic triggers
5. simulator/overlap.py : hard cases: legit users mimicking fraud + fraud users mimicking legit
6. simulator/labels.py : ground truth → labels.parquet, separate from events

Orchestration & Output

7. simulator/run.py : orchestrator: generates subscribers → events → writes to Kinesis + S3

8.simulator/writers.py : helpers: Kinesis PutRecord, S3 batch uploads, DynamoDB loads

Ingestion (AWS side)

9.**Lambda:** activation-ingest : Kinesis sim-activation-stream → S3 raw/activation/
10.**Lambda:** purchase-ingest : Kinesis bundle-purchases-stream → S3 raw/bundles/purchases/

Reference Data

11.**DynamoDB reference loader** : populate subscriber_profile + bundle_catalog from simulator output

Validation

12.**End-to-end test** : make simulate produces events; verify S3, DynamoDB, Kinesis all populated

---

Week 3 : Quality Gate & Silver

Step	What	Why
1	quality_gate/ package skeleton	Where the validation engine lives
2	quality_gate/checks.py — schema validator	Enforce Pydantic contracts
3	quality_gate/checks.py — duplicate check	No repeated event_id
4	quality_gate/checks.py — range check	Non-negative durations, no future timestamps
5	quality_gate/runner.py — orchestrator	Read raw, validate, write silver + quarantine
6	quality_gate/metrics.py — quality metrics	Emit to S3 + SNS threshold alerts
7	quality_gate/glue_wrapper.py — Glue-compatible	Portfolio realism

Step	What	Why
	entry point	
8	`quality_gate/run.py` — CLI + Makefile target	`make quality`
9	Test on the small dataset	Prove the gate works

Week 4 : Gold, Athena & Rules Engine

35. **Silver → Gold Parquet transforms**
36. **Athena DDLs** (activation, revenue, usage, bundles, subscribers)
37. **Rules engine** (SIM swap, IRSF, subscription fraud, bundle abuse)
38. **Candidate alerts to DynamoDB** (`fraud_alerts`)

Week 5 : Model & Scoring Lambda

39. **Feature engineering** (from gold tables)
40. **Model training** (XGBoost + Kaggle secondary validation)
41. **Model artifact to S3** (`telco-fraud-models` bucket)
42. **Scoring Lambda container image**
43. **Cold-start & model size documentation**

Week 6 : Dashboard, Demo & Polish

44. **Dashboard JSON export** (`alerts.json`, `quality.json`)
45. **Static dashboard page** (Netlify-ready)
46. **30-second demo video** (script + recording)

47. **README final pass** (architecture, limitations, how to run)
48. **LinkedIn post draft**